R5.02 — Supervision des réseaux

Compte rendu

TD 1, TD 2 et TP 1

Formation : BUT Réseaux et Télécommunications, semestre 5

Outil : Zabbix 7.0 LTS

Date : 8 octobre 2026

Contenu

- TD 1 : conception d'architecture et ingénierie (exercices 1.1 et 1.2)
- TD 2 : logique d'alerte et escalade (exercices 1 et 2)
- TP 1 : installation de Zabbix, méthode A (paquets Linux sur Debian)

1. TD 1 : Conception d'architecture et ingénierie

1.1 Sujet

Une ETI industrielle a subi une panne critique non détectée du cœur de réseau. La DSI décide de déployer une supervision unifiée avec Zabbix. Le travail demandé est l'ingénierie de cette plateforme.

1.2 Exercice 1.1 : choix d'architecture (réseau WAN)

Énoncé : superviser 30 serveurs d'un site distant depuis le serveur Zabbix central du site principal. L'équipe veut remonter 200 métriques par serveur, toutes les 60 secondes.

Question 1 : bande passante en architecture 100 % centralisée

Le serveur central interroge chaque serveur distant en mode PULL (agents passifs). Une métrique pèse environ 200 octets avec l'encapsulation.

Données : 30 serveurs, 200 métriques par serveur, intervalle de 60 s, 200 octets par métrique.

Calcul :

- Métriques par cycle : 30 × 200 = 6 000
- Métriques par seconde : 6 000 / 60 = 100
- Octets par seconde : 100 × 200 = 20 000 octets/s
- Bits par seconde : 20 000 × 8 = 160 000 bit/s

Résultat : environ 160 kbit/s, soit 0,16 Mbit/s.

Limite : c'est un ordre de grandeur. Il ne compte pas les en-têtes propres à chaque requête PULL. L'énoncé donne déjà 200 octets « avec l'encapsulation ».

Question 2 : risque opérationnel si la fibre tombe (ADSL de 10 Mbit/s partagé)

Le flux représente 0,16 / 10 = 1,6 % de l'ADSL. Il ne sature pas le lien. Le risque n'est donc pas la bande passante.

Le risque majeur est la perte de supervision et de données :

- En mode PULL, le serveur va chercher chaque valeur. Si le lien tombe, la collecte s'arrête et l'historique a des trous.
- Le site distant n'est plus supervisé au moment où un incident se produit.
- Les 30 serveurs passent en « injoignable » en même temps. Cela déclenche une avalanche d'alertes.
- Sur un ADSL instable, les délais de réponse dépassés (timeouts) aggravent le problème.

Question 3 : changement d'architecture, aucune donnée perdue pendant 1 heure

Solution : installer un Zabbix Proxy actif sur le site distant.

Le proxy collecte les 30 serveurs en local. Il envoie ensuite les données au serveur central.

```
30 serveurs  ->  Proxy actif (base SQLite)  ->  ADSL / fibre  ->  Serveur
central
```

Pourquoi cela règle le problème :

- Le proxy garde les données dans sa base locale. Le cours cite le paquet zabbix-proxy-sqlite3.
- Si l'ADSL tombe pendant 1 heure, la collecte continue en local. Rien n'est perdu.
- Quand le lien revient, le proxy renvoie l'historique en attente au serveur central.
- Le proxy compresse le trafic sur le WAN.
- Il doit être actif : il initie lui-même la connexion sortante vers le serveur. Il traverse le NAT et les pare-feux sans ouvrir de port entrant.

> **À vérifier :** le proxy garde les données hors ligne pendant une durée limitée. Le paramètre est ProxyOfflineBuffer. Sa valeur par défaut est de 1 heure d'après ma mémoire. Je ne l'ai pas vérifiée dans la documentation Zabbix. Pour une coupure d'une heure, la marge est faible : il faut augmenter la valeur.

1.3 Exercice 1.2 : dimensionnement matériel (base de données)

Énoncé : 210 hôtes supervisés, en moyenne 250 métriques (items) par équipement, intervalle de rafraîchissement de 60 secondes.

Question 1 : charge de la base en NVPS

Formule du cours 1 : NVPS = (hôtes × éléments) / intervalle.

- Métriques au total : 210 × 250 = 52 500
- Division par l'intervalle : 52 500 / 60 = 875

> **Résultat :** 875 NVPS.

> **Limite :** je n'ai pas retrouvé dans le texte du cours 1 les seuils de classification des NVPS. Je ne les cite donc pas. Le résultat chiffré reste 875 NVPS.

Question 2 : volume disque de l'historique brut sur 30 jours

Données : 875 NVPS, 90 octets par valeur History, 30 jours.

- Secondes en 30 jours : 30 × 24 × 3 600 = 2 592 000 s
- Nombre de valeurs : 875 × 2 592 000 = 2 268 000 000
- Volume en octets : 2 268 000 000 × 90 = 204 120 000 000 octets
- Conversion en Gio : 204 120 000 000 / 1 024³ (soit 1 073 741 824) ≈ 190,1 Gio

> **Résultat :** environ 190 Gio.

> **Attention aux unités :** l'énoncé demande des Gio. On divise donc par 1 024³ et non par 10^9. Avec 10^9, on obtiendrait 204 Go, ce qui répond à une autre unité.

2. TD 2 : Logique d'alerte et escalade

2.1 Exercice 1 : lutte contre la fatigue d'alerte (hystérésis)

Énoncé : le pôle réseau se plaint du bruit généré par la supervision. Le lien WAN principal oscille entre 89 % et 91 % d'occupation en journée. Le déclencheur actuel envoie un email dès que Trafic > 90 %.

Question 1 : nom du phénomène et danger à long terme

Phénomène : le flapping. L'alerte oscille autour du seuil. Chaque passage ouvre ou ferme un incident et envoie un email.

Danger à long terme : la fatigue d'alerte. L'équipe reçoit des centaines de notifications inutiles. Elle finit par les ignorer ou par filtrer les emails. Le jour d'une vraie saturation, l'alerte est noyée dans le bruit et personne ne réagit.

Cause ici : un seul seuil sert à la fois à déclencher et à rétablir l'alerte.

Question 2 : déclencheur avec hystérésis (double seuil)

Solution du cours : dissocier le seuil d'alerte du seuil de rétablissement.

```
Seuil d'alerte          : Trafic > 90 %
Seuil de rétablissement  : Trafic < 70 %

SI état = OK ET Trafic > 90 %
    ALORS état = PROBLEM
          envoyer 1 notification d'ouverture

SI état = PROBLEM ET Trafic < 70 %
    ALORS état = OK
          envoyer 1 notification de fermeture

SINON : ne rien changer (l'état reste stable)
```

- Entre 70 % et 90 %, l'état ne change pas. Un trafic qui oscille entre 89 % et 91 % ne fait plus basculer l'alerte en continu.
- Une fois l'incident ouvert à 91 %, il reste ouvert tant que le trafic ne descend pas sous 70 %. Il y a une seule notification d'ouverture et une seule de fermeture.
- Une vraie saturation est toujours signalée : le seuil d'ouverture reste à 90 %. Rien n'est masqué.
- Option : ajouter une durée, par exemple trafic > 90 % pendant 5 minutes. Cela filtre les pics très courts. Ce n'est pas obligatoire avec l'hystérésis.

> **Remarque :** 70 % est la valeur de l'exemple du cours. Le seuil de rétablissement est un choix d'ingénierie. Une valeur comme 80 % convient aussi, tant qu'elle reste nettement sous 89 %. Ce choix se justifie dans la copie.

2.2 Exercice 2 : matrice d'escalade d'incident

Énoncé : à 02h00, une alerte critique (sévérité Disaster) se déclenche sur le pare-feu du DataCenter principal, qui ne répond plus au ping.

Question 1 : matrice d'escalade à 3 niveaux (T0, T0 + 15 min, T0 + 45 min)

Il est 2 h du matin. Le cours indique que l'email est souvent trop lent pour les astreintes de nuit. Il faut un canal instantané, comme un bot Telegram.

Étape	Cible (qui)	Canal (comment)
T0 (02h00)	Technicien d'astreinte réseau (niveau 1)	Bot Telegram (notification sur smartphone), plus un email pour la trace
T0 + 15 min	Responsable réseau ou manager d'astreinte (niveau 2), si l'incident n'est pas acquitté	Bot Telegram, plus un appel ou un SMS, car le smartphone peut être en silencieux la nuit
T0 + 45 min	DSI ou direction technique (niveau 3), si l'incident est toujours ouvert	Appel téléphonique, plus un email détaillé

> **Source :** seuls T0 (équipe de niveau 1) et T + 15 min (manager de niveau 2) viennent du cours. Le niveau 3 à T0 + 45 min et les canaux appel et SMS sont des choix d'ingénierie. Justification : plus l'incident dure, plus le canal doit être insistant et la cible haut placée.

Un pare-feu de DataCenter en Disaster paralyse probablement beaucoup d'équipements derrière lui. Les dépendances de déclencheurs évitent que chaque serveur envoie sa propre alerte. Un seul incident remonte dans l'escalade.

Question 2 : effet de l'acquittement à T0 + 10 min

Un technicien clique sur « Acquitter l'incident » à T0 + 10 min.

- L'acquittement signale que l'incident est pris en charge.
- À T0 + 15 min, la condition « incident non acquitté » n'est plus vraie. Le niveau 2 n'est pas alerté.
- Le niveau 3 à T0 + 45 min n'est pas alerté non plus, tant que l'incident reste acquitté.
- L'incident reste ouvert (état PROBLEM). Il se ferme seulement quand le déclencheur repasse à OK. Acquitter ne résout pas la panne.

> **À vérifier :** dans Zabbix, ce comportement demande que les étapes 2 et 3 de l'action portent la condition « événement non acquitté ». Je ne l'ai pas vérifié dans la documentation de la version 7.0.

3. TP 1 : Installation et déploiement de Zabbix

3.1 Objectif et sujet

Le TP regroupe deux méthodes pour déployer un serveur Zabbix :

- Méthode A : installation à partir de zéro avec les paquets Linux (prérequis, base de données, services).
- Méthode B : import d'une appliance virtuelle préconfigurée, avec une carte réseau en mode pont.
- Finalisation : accès à l'interface web et validation du serveur.

Méthode réalisée : la méthode A. La partie 2 du TP (appliance) n'a pas été faite. La partie 3 (interface web) est commune aux deux méthodes.

3.2 Environnement

Élément	Valeur
Hyperviseur	Oracle VirtualBox
Système de la VM	Debian 13 (Trixie)
Zabbix	7.0 LTS (dépôt officiel repo.zabbix.com)
Base de données	MariaDB (Debian n'a pas de paquet mysql-server)
Serveur web et PHP	Apache et PHP 8.4
Réseau de la VM	Accès par pont (carte enp0s3)
Langue de l'interface Zabbix	Français

> **Écart avec le sujet :** le PDF du TP date de Debian 9 et de Zabbix 4.4. Plusieurs commandes ont dû être adaptées. Chaque écart est indiqué dans la suite.

3.3 Étape 0 : passage en root

L'utilisateur « user » n'est pas dans le groupe sudo. La commande sudo su est refusée (figure 1). Cela arrive quand un mot de passe root est défini à l'installation.

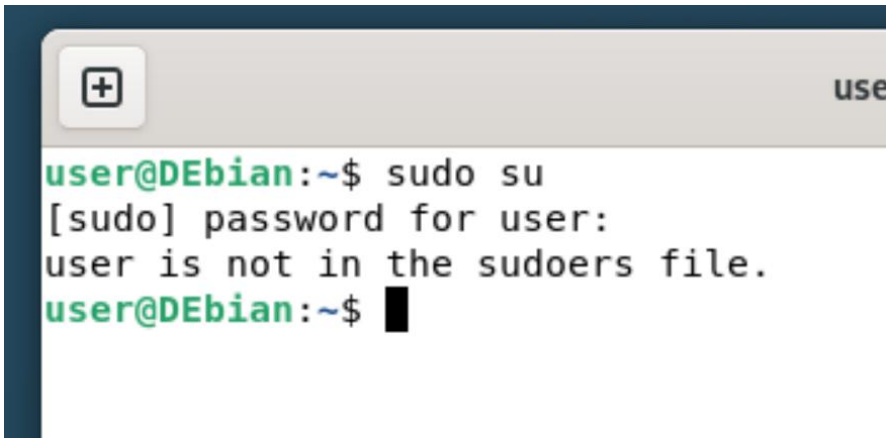


Figure 1 : sudo su refusé : « user is not in the sudoers file »

Solution : passer en root avec su -. Le tiret est important : il charge le PATH complet de root.

```
su -
```

3.4 Étape 1 : préparation du système

Mise à jour des paquets

```
apt update && apt upgrade -y
```

Résultat : tous les paquets sont à jour (0 paquet à mettre à jour).

Installation de la base de données

Commande du TP :

```
apt install mysql-server mysql-client -y
```

Elle échoue sur Debian 13. Le message est « Package mysql-server has no installation candidate » (figure 2). Le système propose mariadb-server et mariadb-client-compat à la place.

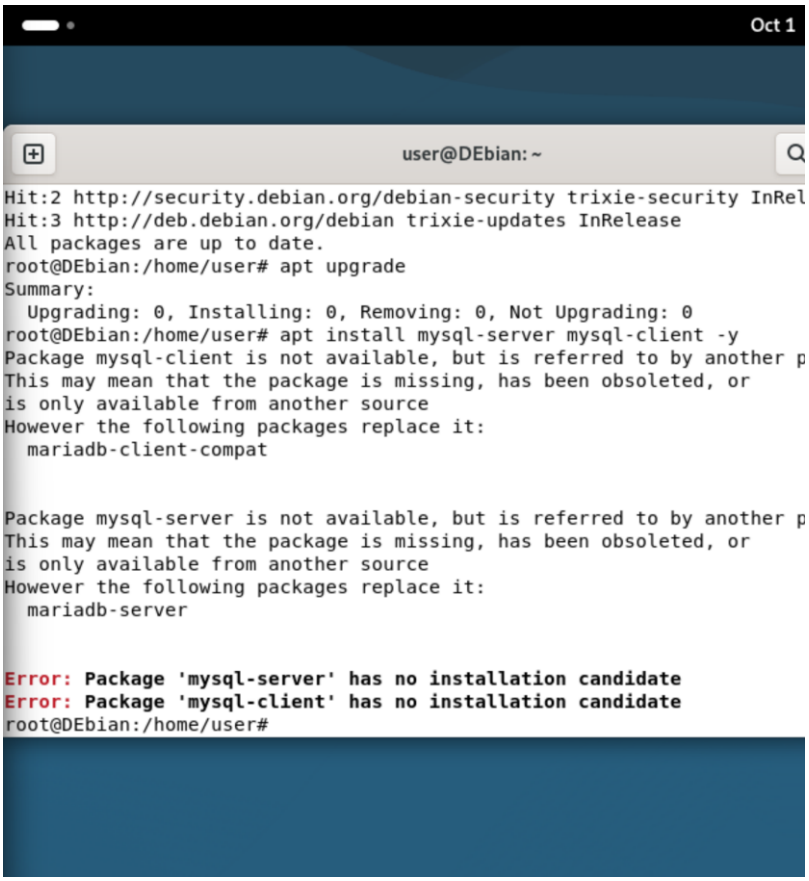


Figure 2 : mysql-server et mysql-client introuvables : MariaDB les remplace

MariaDB est un fork de MySQL. Elle est compatible avec les commandes du TP. Commande réalisée :

```
apt install mariadb-server -y
mariadb --version
```

Le client mariadb est inclus dans mariadb-server. La commande mariadb --version a confirmé sa présence.

Sécurisation de l'installation

```
mariadb-secure-installation
```

Sur MariaDB, la commande s'appelle mariadb-secure-installation. J'ai répondu « Y » à toutes les questions : utilisateurs anonymes supprimés, connexion root distante interdite, base de test supprimée, privilèges rechargés. La réponse « Y » à la question unix_socket fait que root s'authentifie via le compte système root.

3.5 Étape 2 : dépôt Zabbix et installation des composants

Téléchargement du paquet du dépôt

Le TP cite un paquet pour Debian 9 et Zabbix 4.4. Je l'ai remplacé par celui de la version 7.0 pour Debian 13.

```
wget https://repo.zabbix.com/zabbix/7.0/debian/pool/main/z/zabbix-
release/zabbix-release_latest_7.0+debian13_all.deb
```

Le téléchargement a réussi : réponse « 200 OK », fichier de 8,2 Ko (figure 3).

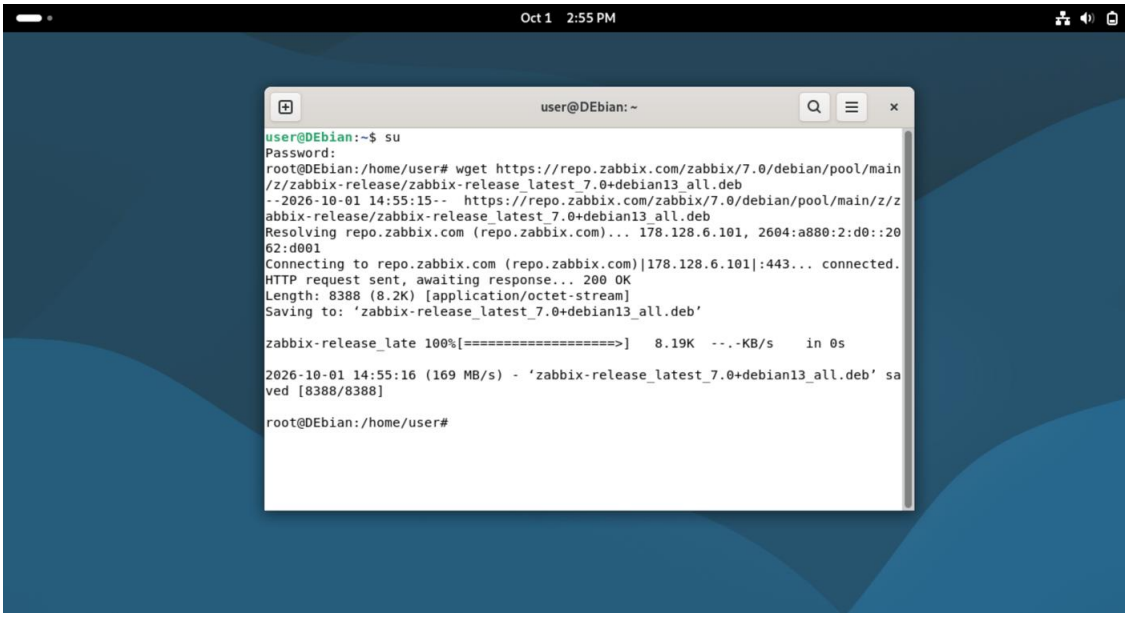


Figure 3 : Téléchargement du paquet zabbix-release pour Debian 13

Installation du paquet du dépôt : première tentative en échec

J'avais ouvert la session root avec su, sans le tiret. La commande dpkg a échoué (figure 4) : ldconfig et start-stop-daemon sont introuvables. Le message final dit que le PATH de root doit contenir /usr/local/sbin, /usr/sbin et /sbin.

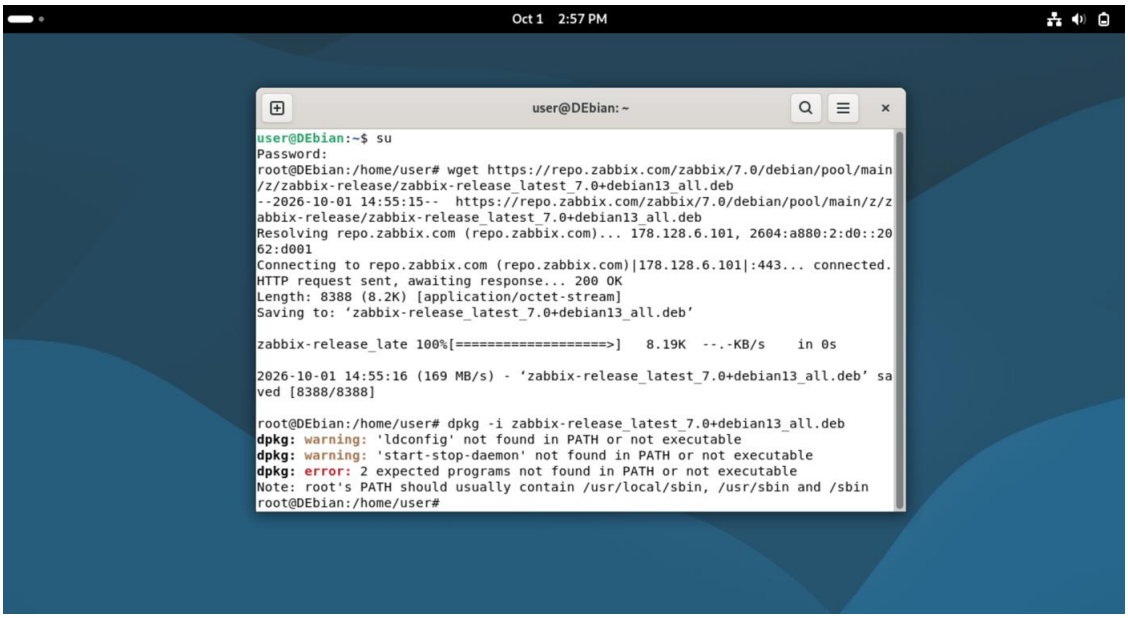


Figure 4 : dpkg en erreur : PATH incomplet après « su » sans tiret

Cause : sans le tiret, su ne charge pas le PATH complet de root. Correction : passer en root avec su -. Le dossier courant devient alors /root et non /home/user. La commande dpkg échoue encore avec « No such file or directory » (figure 5), car le fichier est dans /home/user.

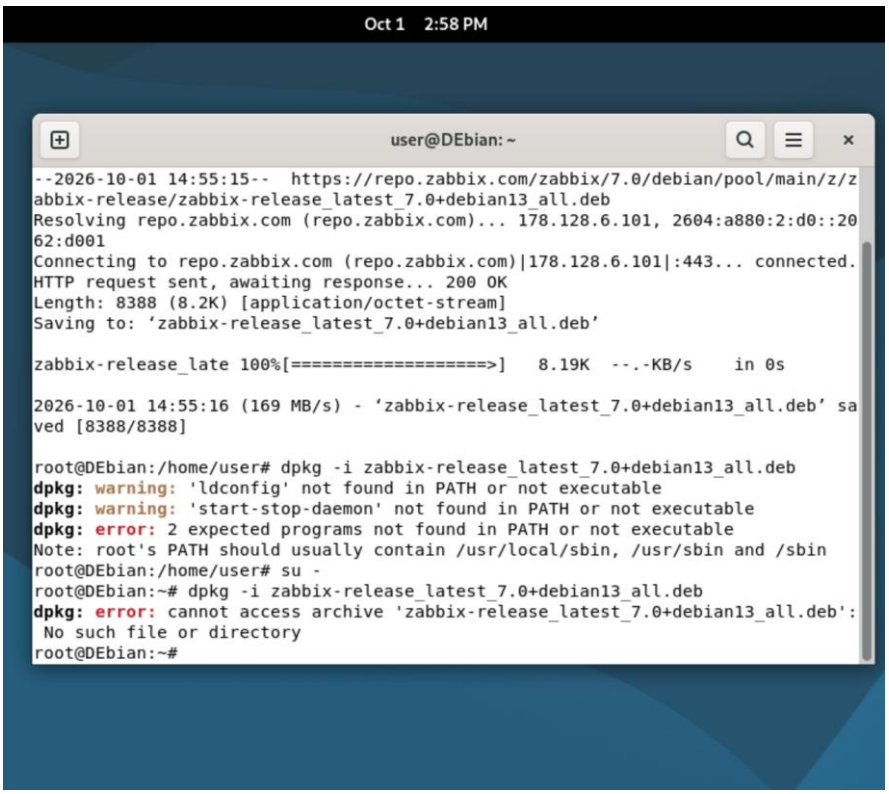


Figure 5 : dpkg en erreur : le fichier est dans /home/user, pas dans /root

Solution : donner le chemin complet du fichier.

```
dpkg -i /home/user/zabbix-release_latest_7.0+debian13_all.deb
apt update
```

L'installation a réussi. Après apt update, la ligne repo.zabbix.com apparaît.

Installation des composants Zabbix

```
apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-conf zabbix-sql-scripts zabbix-agent -y
```

J'ai ajouté le paquet zabbix-sql-scripts par rapport au TP. Avec la version 7.0, il contient le script d'import de la base de données. Apache et PHP s'installent en dépendances. Aucune erreur.

3.6 Étape 3 : création de la base et importation du schéma

Connexion à MariaDB

```
mariadb -uroot -p
```

Le prompt MariaDB [(none)]> s'affiche.

Création de la base et de l'utilisateur

```
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'zabbix';
GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
FLUSH PRIVILEGES;
QUIT;
```

Les quatre requêtes renvoient « Query OK » (figure 6, partie haute). Écarts avec le TP :

- Le TP crée la base en utf8. J'ai utilisé utf8mb4 avec utf8mb4_bin, comme dans le cours 1.
- Le TP crée l'utilisateur et donne les droits en une seule commande (GRANT ... IDENTIFIED BY). Je l'ai séparée en deux commandes : CREATE USER, puis GRANT. Cette forme est la norme et fonctionne sur MariaDB comme sur MySQL 8.

Le mot de passe de l'utilisateur zabbix est « zabbix », comme dans le TP. C'est suffisant pour une VM de TP.

Importation du schéma initial

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mariadb --default-character-set=utf8mb4 -uzabbix -p zabbix
```

Écarts avec le TP :

- Le script n'est plus dans /usr/share/doc/zabbix-server-mysql/create.sql.gz. Il est dans /usr/share/zabbix-sql-scripts/mysql/.
- J'utilise la commande mariadb, native de MariaDB, et l'option --default-character-set=utf8mb4 pour correspondre à l'encodage de la base.

Première tentative : j'ai tapé --default-character-set-=utf8mb4, avec un tiret en trop. MariaDB répond « unknown variable » (figure 6). La base et l'utilisateur étaient bien créés, rien à refaire de ce côté.

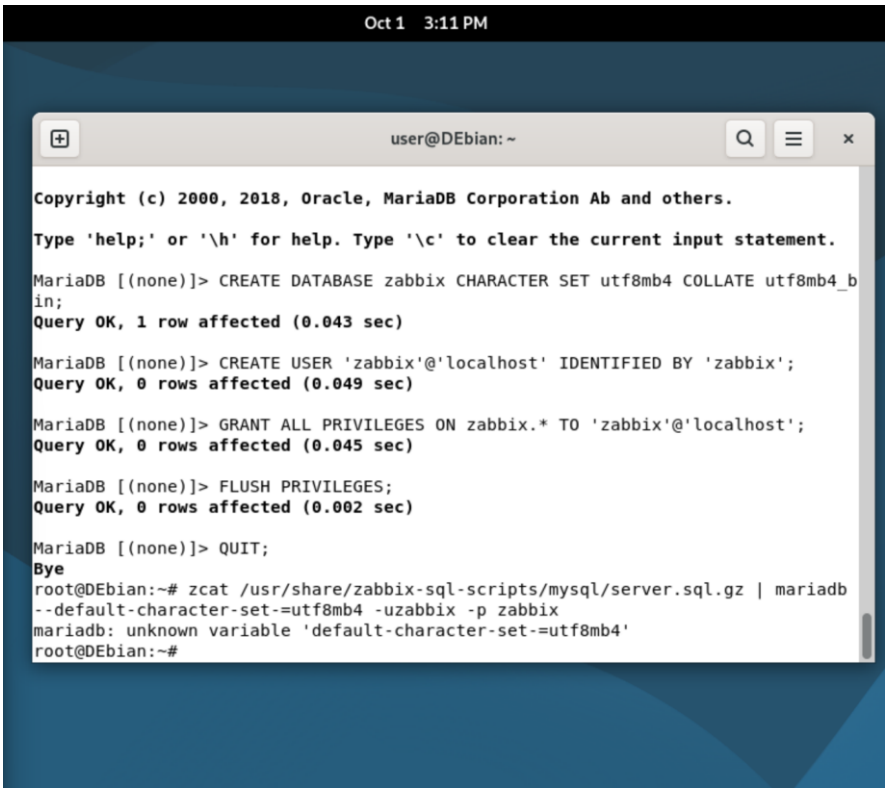


Figure 6 : Création de la base réussie, puis erreur « unknown variable » (tiret en trop)

Deuxième et troisième tentatives (figure 7) : la première est refusée avec « using password: NO » (aucun mot de passe saisi). La suivante est refusée avec « using password: YES » (mot de passe faux). La dernière a réussi : aucune erreur, le prompt revient.

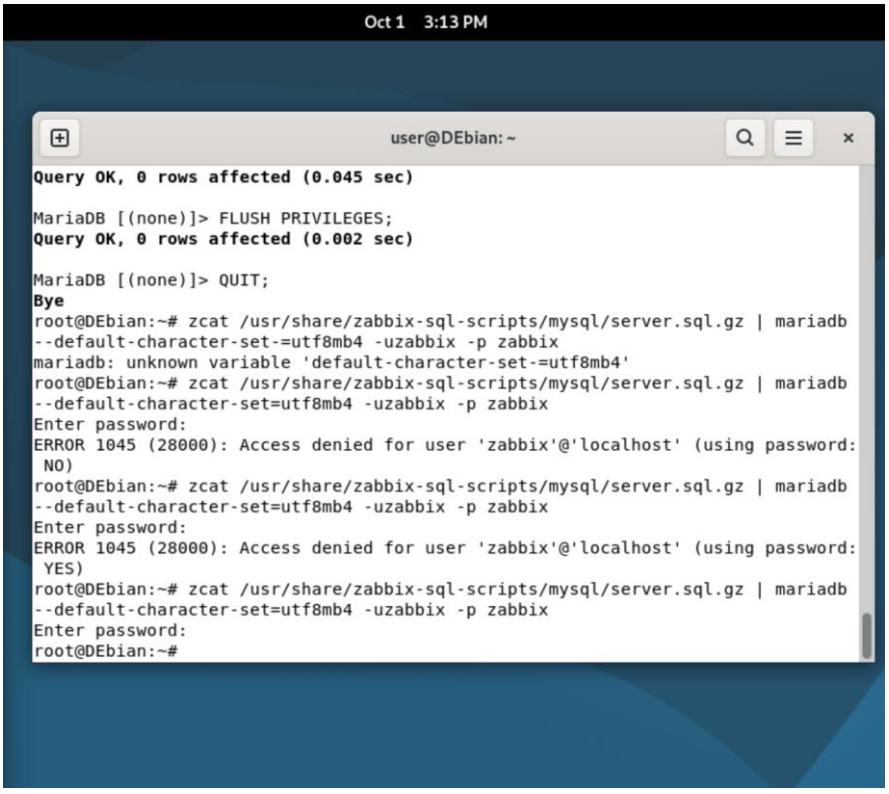


Figure 7 : Tentatives d'import : erreurs 1045 sur le mot de passe, puis réussite

Vérification de l'import

```
mariadb -uzabbix -p -e "SELECT COUNT(*) FROM information_schema.tables WHERE
table_schema='zabbix';"
```

Résultat : 203 tables dans la base zabbix (figure 8). L'import a réussi.

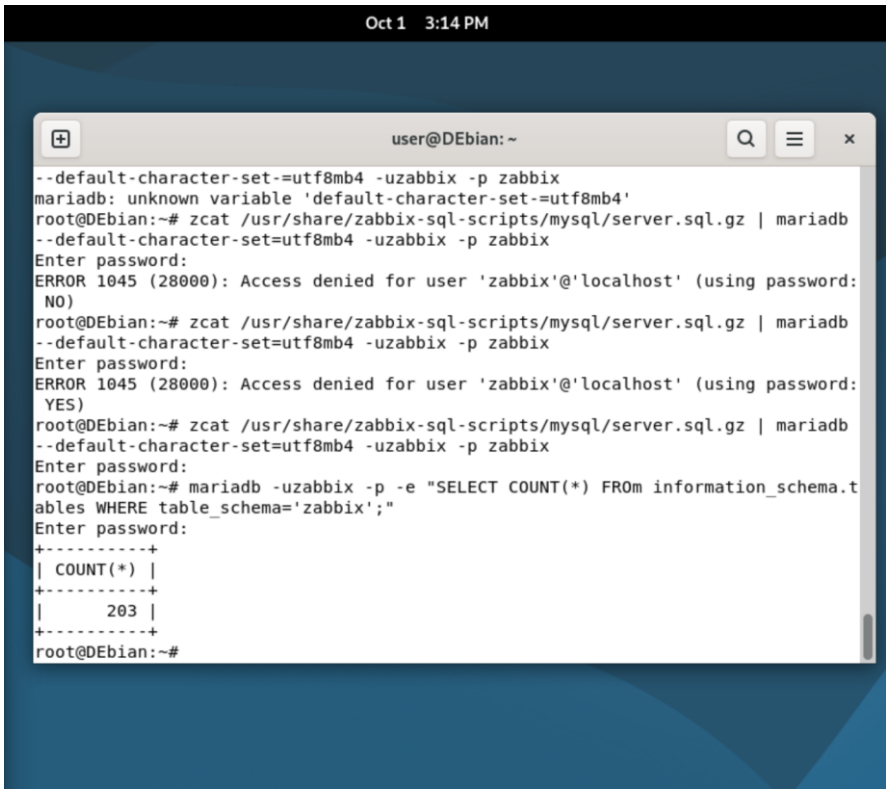


Figure 8 : Vérification : 203 tables dans la base zabbix

3.7 Étape 4 : configuration et démarrage des services

Liaison du serveur à la base de données

```
nano /etc/zabbix/zabbix_server.conf
```

Dans le fichier, j'ai réglé :

```
DBUser=zabbix
DBPassword=zabbix
```

DBHost et DBName ont par défaut les valeurs localhost et zabbix. Elles correspondent à la configuration. Le TP demande de les écrire. Ce n'est pas obligatoire.

Erreur de ma part : j'avais d'abord écrit DBPassword=user (figure 9). Or le mot de passe de l'utilisateur MariaDB zabbix est « zabbix ». Avec « user », le serveur serait refusé par la base (« Access denied »). Je l'ai corrigé, puis contrôlé avec grep.

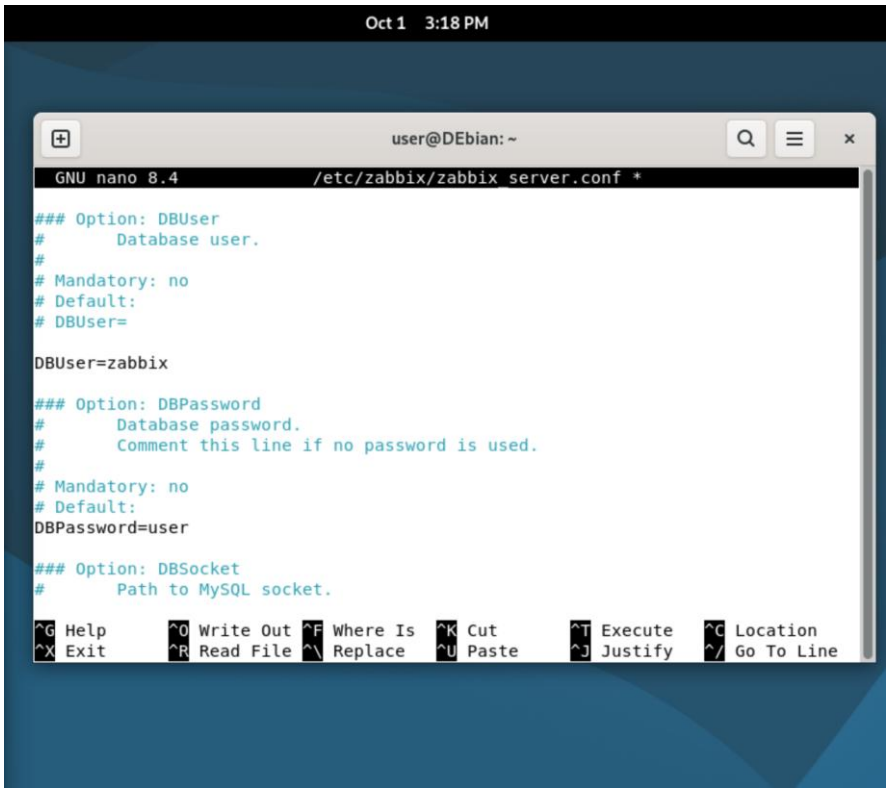


Figure 9 : zabbix_server.conf : DBPassword=user, valeur fausse corrigée ensuite

```
grep ^DB /etc/zabbix/zabbix_server.conf
```

La commande affiche DBUser=zabbix et DBPassword=zabbix.

Fuseau horaire PHP

Le TP demande de modifier /etc/apache2/conf-enabled/zabbix.conf. Sur Debian 13 avec PHP 8.4, ce fichier ne contient pas de ligne timezone. J'ai cherché ailleurs. La ligne date.timezone est dans le fichier php.ini utilisé par Apache :

```
nano /etc/php/8.4/apache2/php.ini
```

Ligne réglée (sans point-virgule au début, sinon elle reste inactive) :

```
date.timezone = Europe/Paris
```

Démarrage et activation des services

```
systemctl restart zabbix-server zabbix-agent apache2
systemctl enable zabbix-server zabbix-agent apache2
systemctl is-active zabbix-server zabbix-agent apache2
```

Aucune erreur. La dernière commande affiche « active » pour les trois services.

3.8 Étape 5 : réseau et accès à l'interface web

Passage de la carte réseau en mode pont

La commande ip a montrait l'adresse 10.0.2.15 sur enp0s3. C'est l'adresse du mode NAT, la valeur par défaut de VirtualBox. Dans ce mode, le PC ne peut pas joindre la VM.

Le mode pont est demandé par le TP (partie 2, étape 6). Marche suivie :

- Éteindre la VM avec la commande poweroff.
- Dans VirtualBox : Configuration, onglet Réseau, carte 1, mode d'accès « Accès par pont ».
- Choisir la carte physique du PC connectée à Internet, puis valider.
- Redémarrer la VM et relancer ip a.

La VM a d'abord reçu l'adresse 172.27.171.87/22 (réseau privé de la plage 172.16.0.0 à 172.31.255.255). Après un changement de réseau côté PC, l'adresse est devenue 172.20.10.10. Cette adresse peut changer à chaque redémarrage de la VM.

Accès depuis le navigateur du PC

```
http://172.20.10.10/zabbix
```

La page « Welcome to Zabbix » s'affiche.

3.9 Étape 6 : assistant d'installation web et validation

Page de l'assistant	Réglage
Prérequis	Toutes les lignes sont OK, y compris le fuseau horaire PHP et la prise en charge de la base de données
Connexion à la base	Type MySQL (valable pour MariaDB), hôte localhost, base zabbix, utilisateur zabbix, mot de passe zabbix
Chiffrement TLS de la base	Non chiffré : la connexion passe par un fichier socket local, ce qui est normal avec localhost
Paramètres du serveur	Nom du serveur choisi librement, fuseau horaire par défaut Europe/Paris, thème au choix
Résumé	Installation terminée sans erreur

Connexion : identifiant Admin (le A majuscule est obligatoire), mot de passe zabbix.

> **Résultat :** dans le widget « Informations système » du tableau de bord, la ligne « Le serveur Zabbix est en cours d'exécution » affiche « Oui ». Le TP est validé.

J'ai gardé le mot de passe par défaut pour ce TP. La VM étant en mode pont, elle est joignable depuis le réseau. Ce choix est acceptable pour une VM de TP de courte durée. Pour une utilisation plus longue, il faut changer le mot de passe.

3.10 Récapitulatif des écarts entre le sujet du TP et la réalisation

Point	Sujet du TP	Réalisé
Base de données	mysql-server	mariadb-server (Debian n'a pas mysql-server)
Sécurisation	mysql_secure_installation	mariadb-secure-installation
Dépôt Zabbix	Paquet 4.4 pour Debian 9	Paquet 7.0 pour Debian 13
Paquets installés	Sans zabbix-sql-scripts	Avec zabbix-sql-scripts
Encodage de la base	utf8 / utf8_bin	utf8mb4 / utf8mb4_bin

Point	Sujet du TP	Réalisé
Création de l'utilisateur	GRANT ... IDENTIFIED BY en une commande	CREATE USER puis GRANT
Script d'import	/usr/share/doc/zabbix-server-mysql/create.sql.gz	/usr/share/zabbix-sql-scripts/mysql/server.sql.gz
Fuseau horaire PHP	/etc/apache2/conf-enabled/zabbix.conf	/etc/php/8.4/apache2/php.ini
Réseau de la VM	Mode pont (pour l'appliance)	Mode pont, appliqué aussi à la méthode A

3.11 Erreurs rencontrées et leçons

Erreur	Cause	Correction
sudo su refusé	L'utilisateur n'est pas dans le groupe sudo	Utiliser su -
dpkg : ldconfig et start-stop-daemon introuvables	su sans tiret : PATH incomplet	Utiliser su -
dpkg : No such file or directory	Après su -, le dossier courant est /root	Donner le chemin complet /home/user/...
unknown variable sur l'import	Tiret en trop dans --default-character-set	Recopier la commande exactement
Erreur 1045 Access denied	Mot de passe absent ou faux	Saisir le bon mot de passe de l'utilisateur zabbix
DBPassword=user	Confusion entre le nom d'utilisateur et le mot de passe	DBPassword=zabbix

Aucune capture n'a été prise pour l'interface web (assistant et tableau de bord). Le résultat est décrit dans la section 3.9.